AWS NETWORKING / PROJECT 01

VPC Peering

Connecting two VPCs and proving private connectivity

Project report: EC2, custom route tables, security groups and a peering connection

2 VPCs	3 subnets	1 peering connection	3 EC2 instances

PROJECT DETAILS

EWB Courses | AWS Project

Region: Asia Pacific (Sydney), ap-southeast-2 | Completed: 29 October 2026

Contents

1. Objective 3

2. Architecture and address plan 3

3. Resource inventory 5

4. Implementation 5

5. Security design 11

6. Testing and verification 11

7. Forward path and return path 12

8. Key concepts and questions answered 13

9. Troubleshooting checklist 13

10. Cost, quotas and limits 13

11. Choosing peering or an alternative 13

12. AWS CLI equivalent 14

13. Hardening and next steps 14

14. Cleanup plan 14

15. What I learned 14

Project at a glance

Item	Detail
Region	Asia Pacific (Sydney), ap-southeast-2
VPC A	ewb-vpc-a, 10.10.0.0/16: one public and one private subnet, and the only VPC with an internet gateway
VPC B	ewb-vpc-b, 10.20.0.0/16: one private subnet, no internet gateway
Peering connection	ewb-a-b-peer (pcx-00647a9dc9f82f931), status Active
Compute	ewb-bastion (public), ewb-app-a and ewb-app-b (private, no public IPv4)
Security	Inbound rules limited to single hosts (/32); nothing open to 0.0.0.0/0
Evidence	Forward and reverse pings captured; items I did not capture are marked as such

1. Objective

The goal was to connect two separate Virtual Private Clouds with a VPC peering connection so that workloads in each can talk to each other over private IPv4 addresses, without using the public internet. The build also had to show that peering alone is not enough: non-overlapping CIDR ranges, explicit route table associations, routes in both directions and tightly scoped inbound rules all matter.

This report shows how I:

- planned two non-overlapping VPC and subnet ranges,
- created public and private subnets with their own route tables,
- requested, accepted and activated a peering connection and added routes on both sides,
- launched EC2 instances and verified private connectivity with ping.

2. Architecture and address plan

VPC A holds a public subnet (the bastion) and a private subnet (App-A) and is the only VPC with an internet gateway. VPC B holds a single private subnet (App-B) and has no internet gateway at all. The peering connection links the two VPCs.

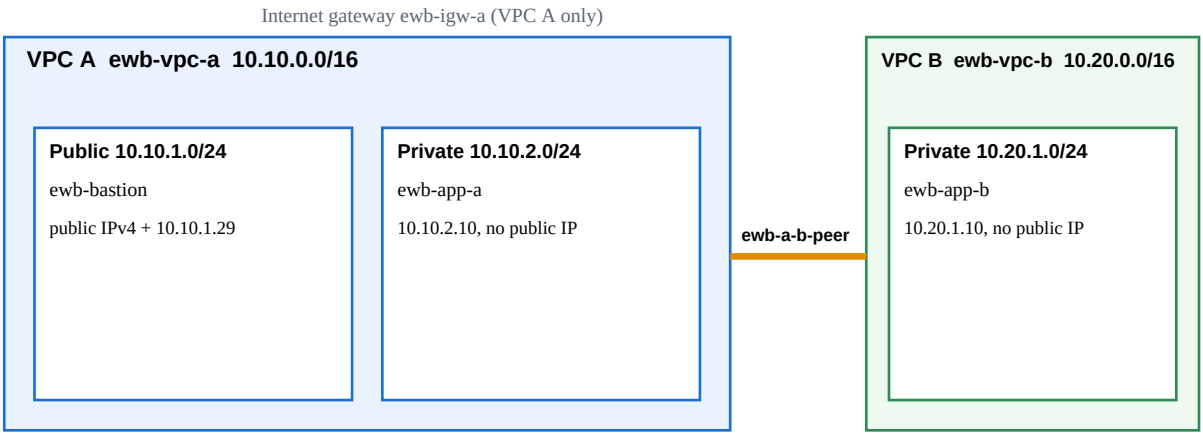


Diagram. Architecture (conceptual). Peering does not give VPC B access to VPC A's internet gateway.

Address plan

Resource	Placement	IPv4 CIDR
ewb-vpc-a	VPC A	10.10.0.0/16
ewb-public-subnet-a	VPC A, public subnet	10.10.1.0/24
ewb-private-subnet-a	VPC A, private subnet	10.10.2.0/24
ewb-vpc-b	VPC B	10.20.0.0/16
ewb-b-private	VPC B, private subnet	10.20.1.0/24

The two VPC ranges do not overlap, which is a hard requirement: AWS refuses to create a peering connection between VPCs with overlapping CIDRs. Each /24 subnet sits inside its own VPC range.

3. Resource inventory

IDs below are the ones shown in the AWS console for this build.

Resource	Name	ID / detail
VPC A	ewb-vpc-a	vpc-0a7a8c5dd60b0227d (10.10.0.0/16)
VPC B	ewb-vpc-b	vpc-0701af0f2c948761a (10.20.0.0/16)
Subnet	ewb-public-subnet-a	subnet-0a9c10e6e2f6a2925 (10.10.1.0/24)
Subnet	ewb-private-subnet-a	subnet-0630265e8e8573cff (10.10.2.0/24)
Subnet	ewb-b-private	subnet-085ed6dadaa1bb47d (10.20.1.0/24)
Internet gateway	ewb-igw-a	attached to VPC A (ID not captured)
Route table	ewb-rt-a-public	rtb-0d577742a22ba4836
Route table	ewb-rt-a-private	rtb-008e95adf761125aa
Route table	ewb-rt-b-private	rtb-035c7551d259e1771
Peering	ewb-a-b-peer	pcx-00647a9dc9f82f931, Active
EC2	ewb-bastion	i-040b56281f2f321bc, 10.10.1.29 + public IPv4
EC2	ewb-app-a	i-011efa5d7a099ef78, 10.10.2.10, no public IP
EC2	ewb-app-b	i-0ac3cd2df6db5a891, 10.20.1.10, no public IP

I let AWS assign the bastion's private IP (10.10.1.29), so every rule that refers to the bastion uses that address, not 10.10.1.10. Everything was tagged Project = EWB-VPC-Peering and Student = (my name) to make cleanup easy.

Build order

Each resource depends on the one before it: VPCs, subnets, internet gateway, route tables, peering connection, peer routes, EC2 instances, then security group rules.

4. Implementation

Step 1: Create the VPCs

I created ewb-vpc-a (10.10.0.0/16) and ewb-vpc-b (10.20.0.0/16) as VPC-only, so I could control subnets and route tables myself.

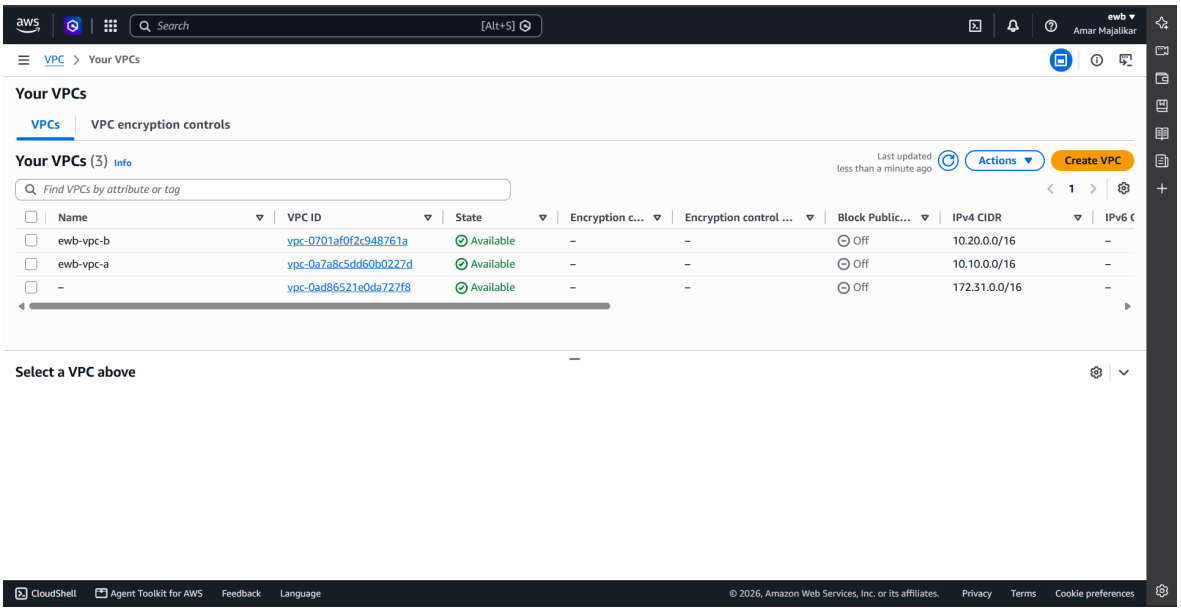


Figure 1. Both VPCs with their IDs and CIDR ranges.

Step 2: Create the subnets

Two subnets went into VPC A and one into VPC B, all in the same Availability Zone (ap-southeast-2a) so traffic between them does not cross zones.

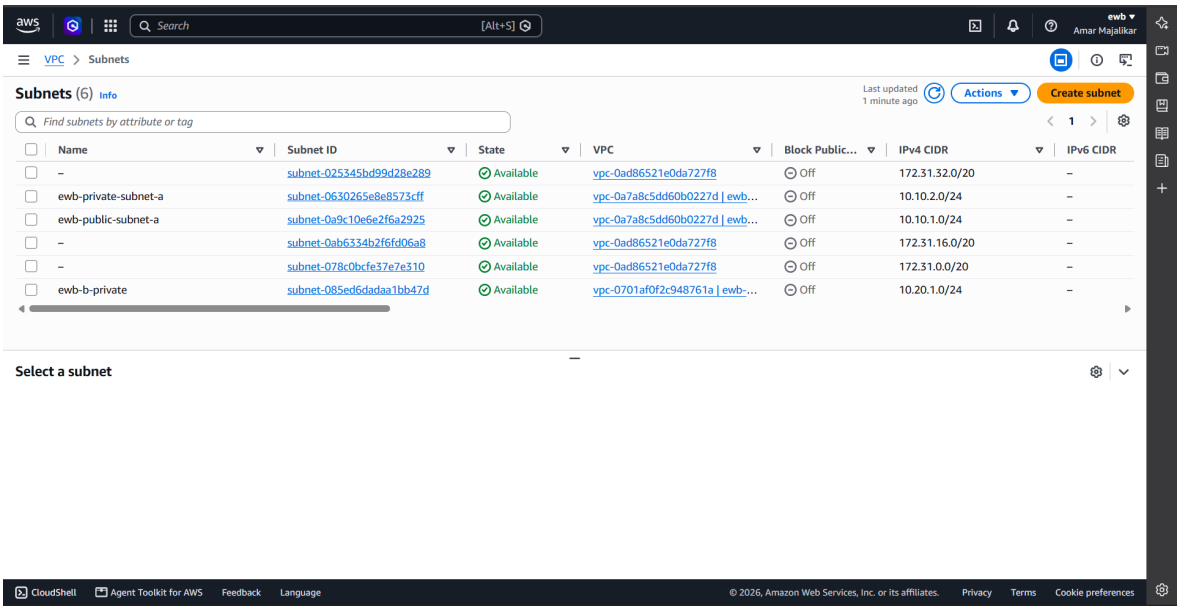


Figure 2. Subnet CIDRs: 10.10.1.0/24, 10.10.2.0/24 and 10.20.1.0/24.

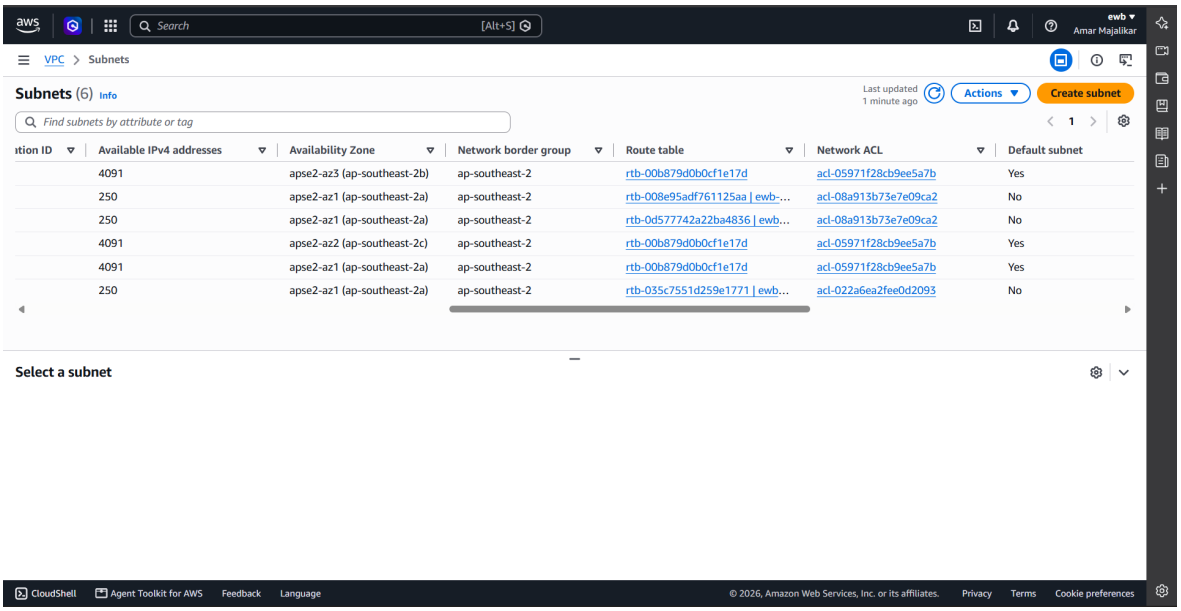


Figure 3. All three subnets in ap-southeast-2a, each with its own route table.

Step 3: Internet gateway and route tables

An internet gateway (ewb-igw-a) was attached to VPC A only. I created custom route tables and associated each subnet explicitly, instead of relying on the main route table. The public route table sends 0.0.0.0/0 to the gateway; the private route tables have no internet route.

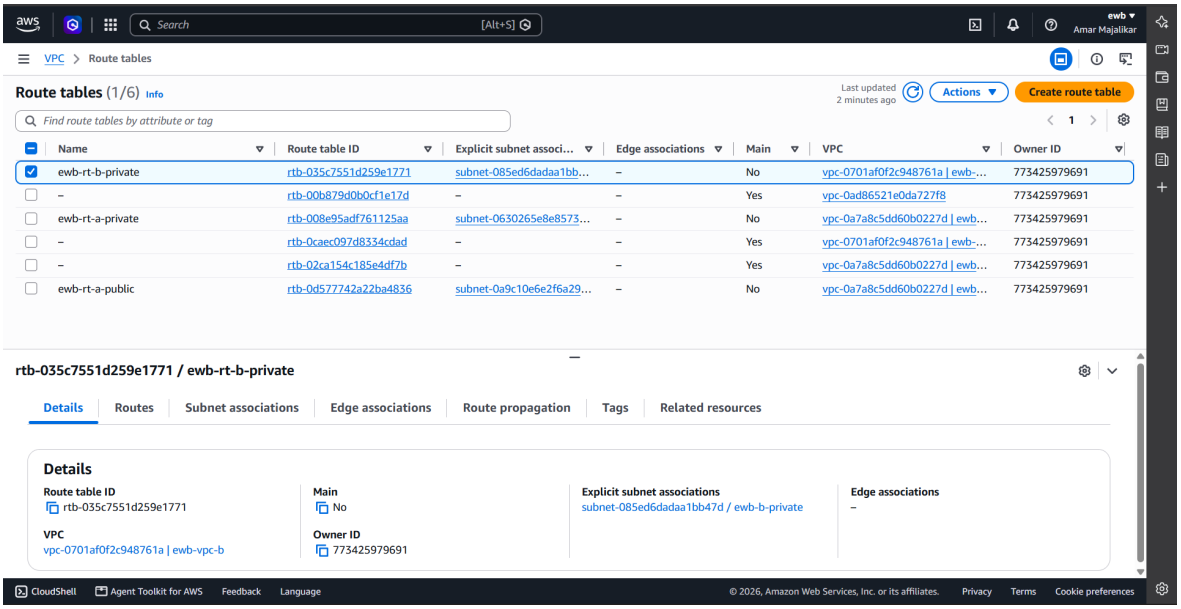


Figure 4. Route tables ewb-rt-a-public, ewb-rt-a-private and ewb-rt-b-private with explicit subnet associations.

I did not capture the Routes tab of each table. The routes below are the ones I configured:

Route table	Destination	Target	Purpose
ewb-rt-a-public	10.10.0.0/16	local	Bastion to App-A
ewb-rt-a-public	0.0.0.0/0	ewb-igw-a	Bastion internet access
ewb-rt-a-public	10.20.0.0/16	pcx-00647a9dc9f82f931	Bastion to VPC B
ewb-rt-a-private	10.10.0.0/16	local	VPC A local route
ewb-rt-a-private	10.20.0.0/16	pcx-00647a9dc9f82f931	App-A to VPC B

Route table	Destination	Target	Purpose
ewb-rt-b-private	10.20.0.0/16	local	VPC B local route
ewb-rt-b-private	10.10.0.0/16	pcx-00647a9dc9f82f931	Return path to VPC A

Step 4: Peering connection

I requested ewb-a-b-peer with VPC A as requester and VPC B as accepter. Both VPCs are in my account and Region, but I still had to accept the request myself before the status became Active.

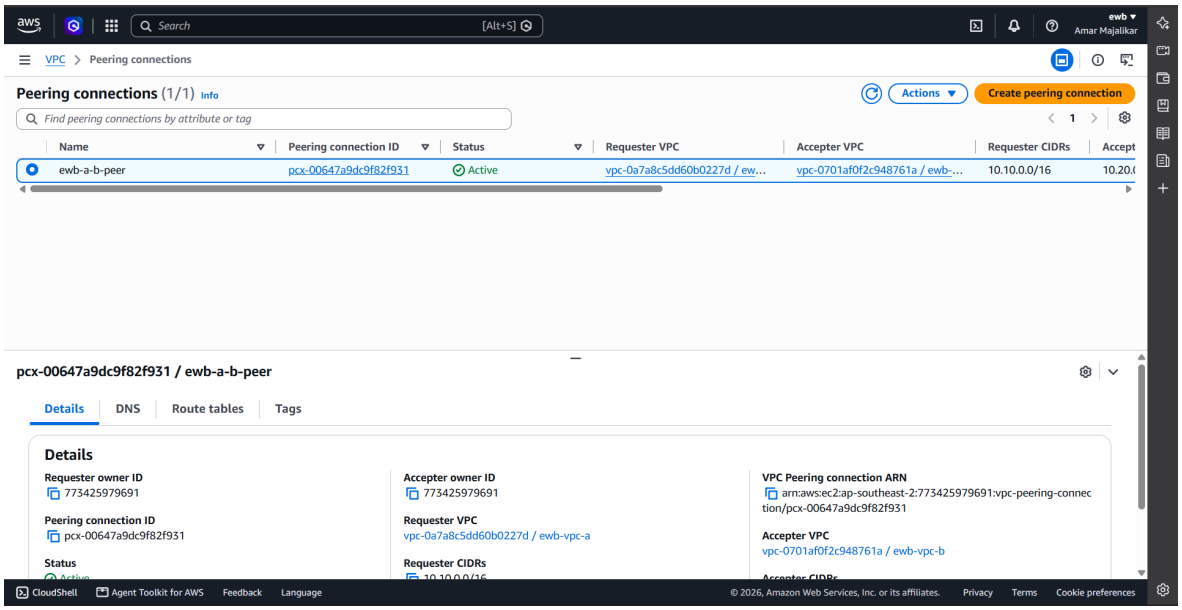


Figure 5. ewb-a-b-peer is Active: requester vpc-0a7a8c5dd60b0227d (10.10.0.0/16), accepter vpc-0701af0f2c948761a (10.20.0.0/16).

Step 5: Launch the EC2 instances

I launched one bastion in the public subnet and one private instance in each private subnet. App-A and App-B were launched without a public IPv4 address.

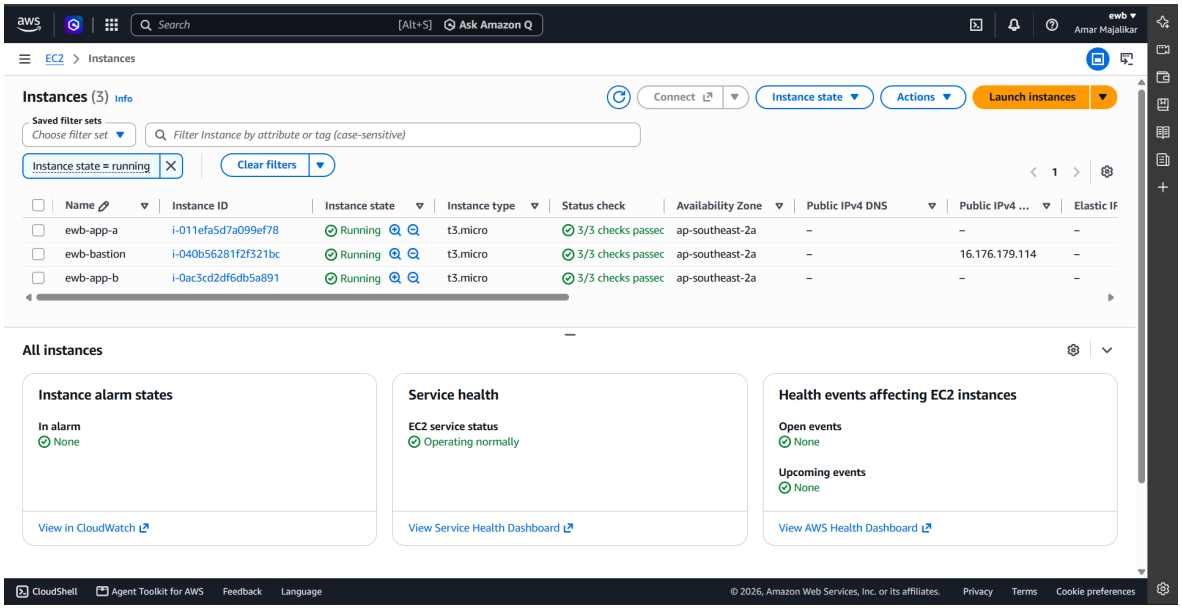


Figure 6. All three instances running with 3/3 status checks; only ewb-bastion has a public IPv4.

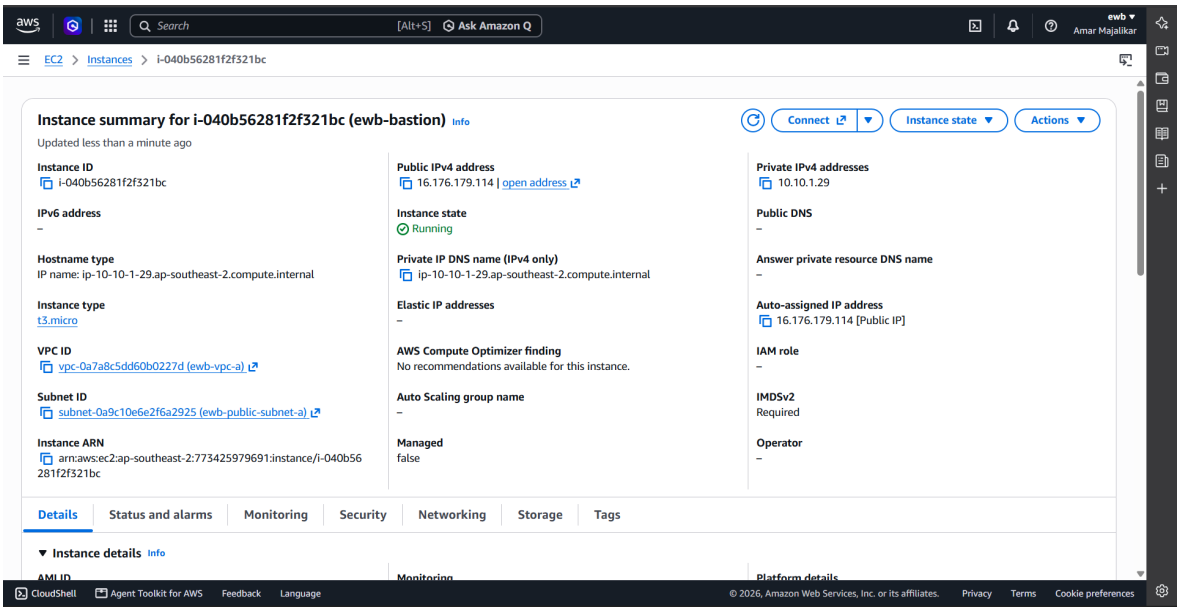


Figure 7. ewb-bastion: public IPv4 16.176.179.114, private IPv4 10.10.1.29, in ewb-public-subnet-a.

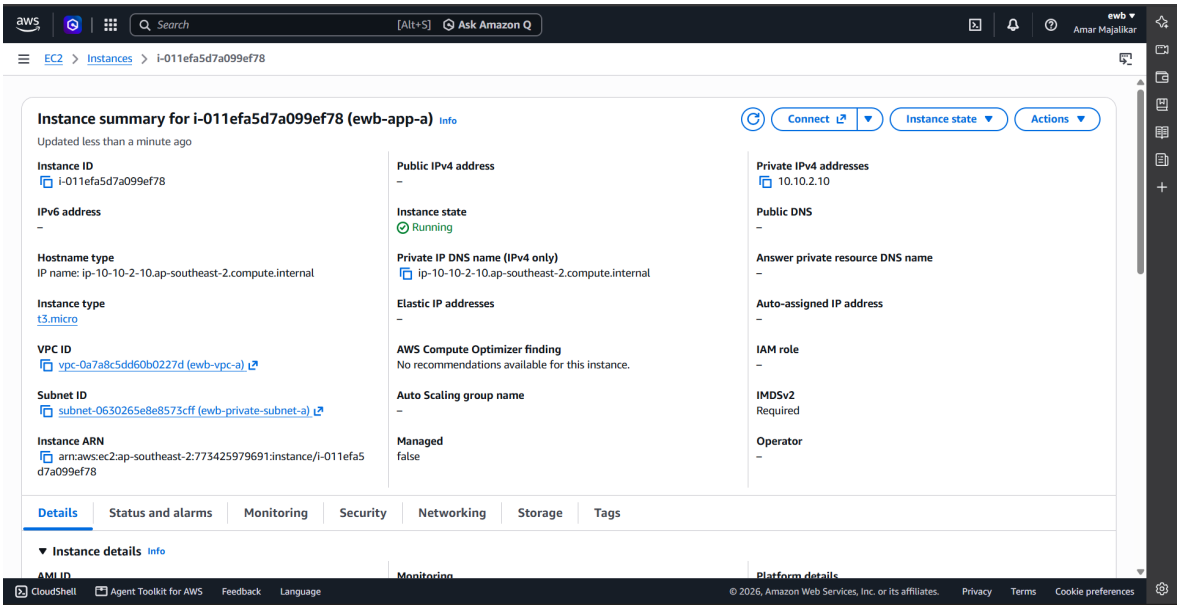


Figure 8. ewb-app-a: no public IPv4, private IPv4 10.10.2.10, in ewb-private-subnet-a.

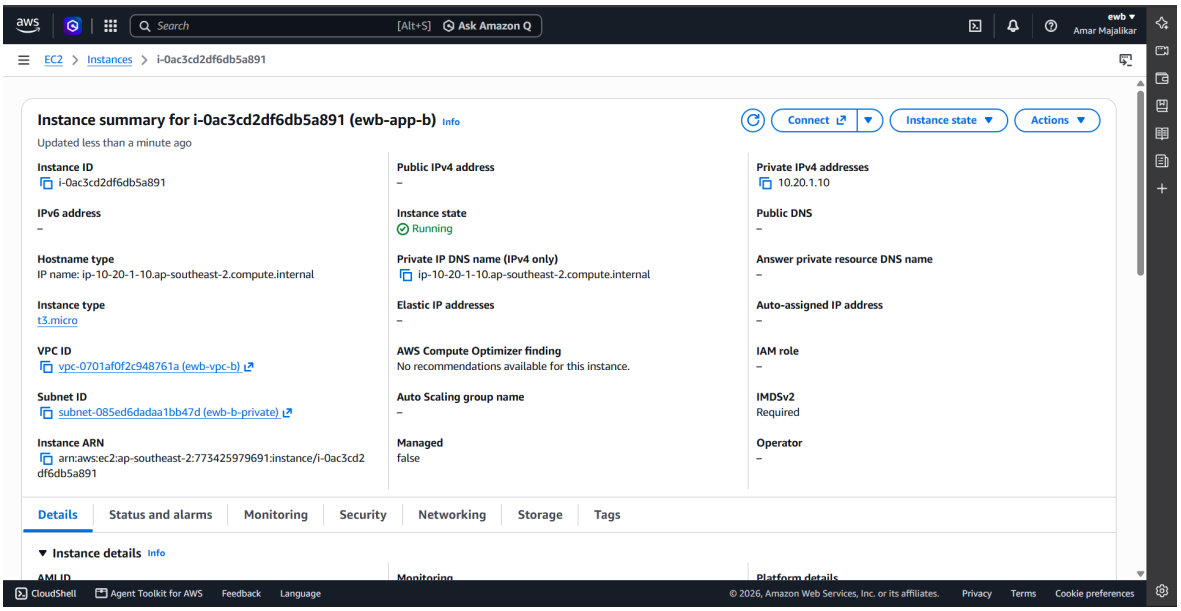


Figure 9. ewb-app-b: no public IPv4, private IPv4 10.20.1.10, in ewb-b-private.

Step 6: Security group rules

Each rule allows one specific host. Outbound traffic was left at the default allow-all.

Security group	VPC	Type / port	Source
ewb-sg-bastion	A	SSH / TCP 22	103.217.82.81/32 (my public IPv4)
ewb-sg-app-a	A	SSH / TCP 22	10.10.1.29/32 (bastion private IP)
ewb-sg-app-a	A	All ICMP - IPv4	10.20.1.10/32 (App-B)
ewb-sg-app-b	B	SSH / TCP 22	10.10.1.29/32 (bastion private IP)
ewb-sg-app-b	B	ICMP	10.10.2.10/32 (App-A)

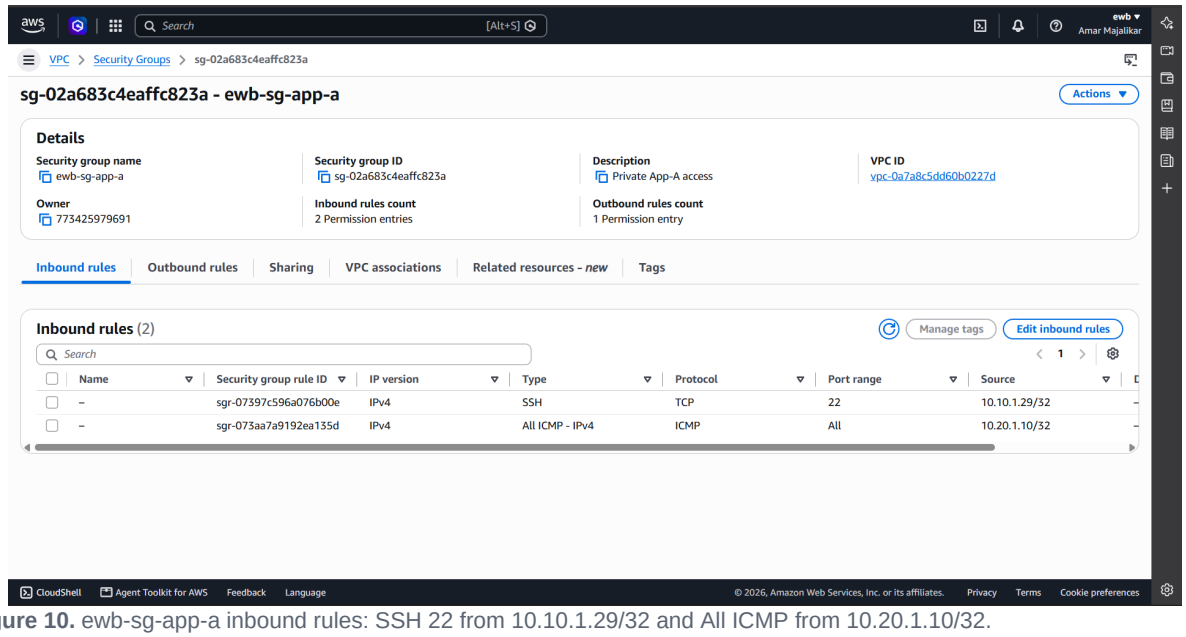


Figure 10. ewb-sg-app-a inbound rules: SSH 22 from 10.10.1.29/32 and All ICMP from 10.20.1.10/32.

I did not capture screenshots of the inbound rules of ewb-sg-bastion and ewb-sg-app-b. One thing I would tighten: the App-A ICMP rule is "All ICMP - IPv4" and should be Echo request only.

5. Security design

A packet has to pass several independent layers before it reaches a private instance. Removing any one layer stops the traffic:

- **Routing:** the source subnet needs a route to the destination CIDR through the peering connection.
- **Peering status:** the connection must be Active.
- **Destination security group:** must allow the protocol from the source host.
- **Network ACLs:** the defaults allow everything; custom ACLs would need matching rules.

Design choices

Choice	What I did	Why it matters
Non-overlapping CIDRs	10.10.0.0/16 and 10.20.0.0/16	Peering cannot be created between overlapping ranges
Explicit associations	Each subnet tied to its own route table	No accidental use of the main route table
Internet gateway	Only on VPC A, public route table only	Private hosts cannot be reached from the internet
Single-host rules	/32 sources, nothing on 0.0.0.0/0	Least privilege
No public IP on private hosts	App-A and App-B have none	Only the bastion is exposed

6. Testing and verification

All tests use private destination IPs. The table shows the result I captured for each test; where I did not capture a result, it says so.

Test	Where / command	Expected	My result
01 Baseline (before peering)	On App-A: ping -c 4 10.20.1.10	No replies	Not captured
02 Forward path	On App-A: ping -c 4 10.20.1.10	4 replies, 0% loss	4 replies, 0% loss (Figure below)
03 Reverse path	On App-B: ping -c 4 10.10.2.10	4 replies, 0% loss	4 replies, 0% loss (Figure below)
04 Private SSH	Laptop: ssh ewb-app-b via ProxyJump	Shell on App-B through the bastion	Not captured via ProxyJump (see note)
05 Isolation	EC2 details of App-A and App-B	No public IPv4 on either	Confirmed in the instance screenshots

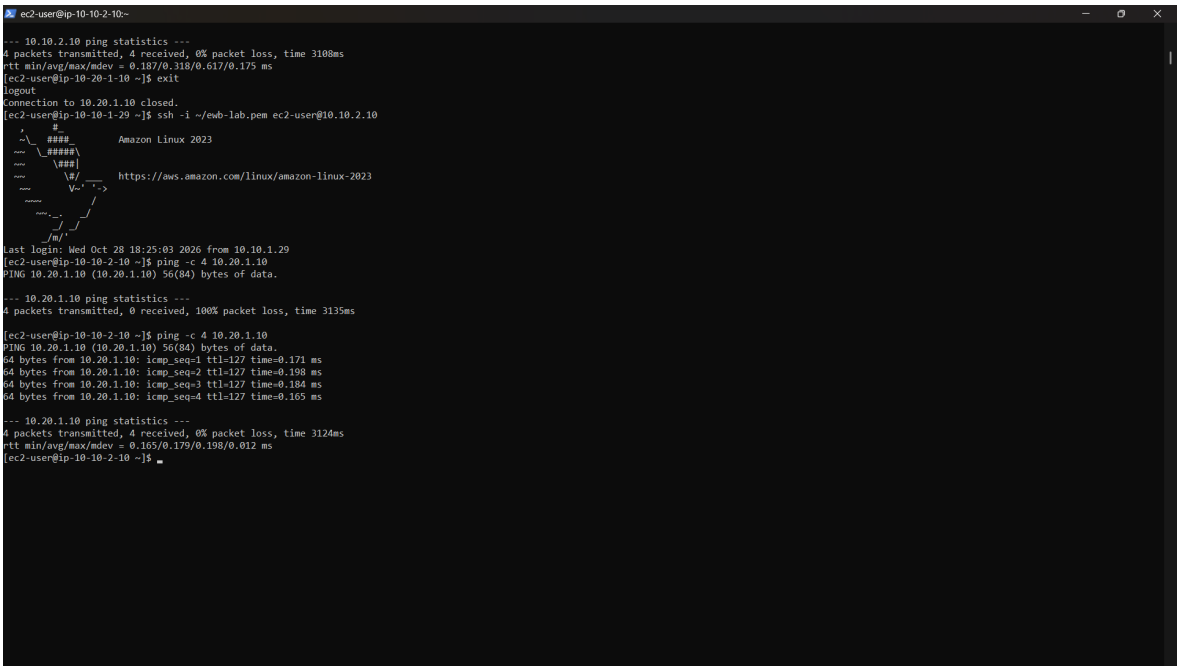


Figure 11. Terminal: reverse ping from App-B (ip-10-20-1-10) to 10.10.2.10 with 4 of 4 replies; then on App-A (ip-10-10-2-10) the forward ping to 10.20.1.10, one run with 4 of 4 lost and a later run with 4 of 4 replies.

How I reached the hosts. ProxyJump from my Windows laptop failed with "Permission denied (publickey)", so I connected to the bastion directly and hopped from there to App-A and App-B using a copy of the key file on the bastion. The workbook says to keep the private key on the laptop and use ProxyJump, so this does not meet that requirement, and I did not capture test 04.

Controlled ICMP failure. I did not capture the controlled failure test (removing only the App-B inbound ICMP echo rule from 10.10.2.10/32, then restoring it), so I have no output to show. What it demonstrates: with the rule removed, pings from App-A to App-B get no replies even though the peering is Active and both routes exist; after restoring the exact rule the replies resume, and SSH stays untouched throughout. Routes decide whether a packet can find its way across; the destination's security group decides whether it is let in.

7. Forward path and return path

Forward path (App-A to App-B). When App-A (10.10.2.10) pings App-B (10.20.1.10), the packet is looked up in ewb-rt-a-private, matches 10.20.0.0/16 and goes through the peering connection. In VPC B, ewb-sg-app-b accepts it because it allows ICMP echo requests from 10.10.2.10/32.

Return path (App-B back to App-A). App-B's reply is looked up in ewb-rt-b-private, where 10.10.0.0/16 sends it back through the peering connection. Security groups are stateful, so the reply is allowed automatically and needs no extra inbound rule.

Source	Route table used	Matching route	Result
App-A to 10.20.1.10	ewb-rt-a-private	10.20.0.0/16 -> pcx	Delivered over peering
App-B to 10.10.2.10	ewb-rt-b-private	10.10.0.0/16 -> pcx	Delivered over peering
App-A to 8.8.8.8	ewb-rt-a-private	none	Dropped (no internet route)
Bastion to 8.8.8.8	ewb-rt-a-public	0.0.0.0/0 -> igw	Reaches the internet

8. Key concepts and questions answered

1. Why is the return route needed? Peering does not add routes on its own. If VPC B has no route to 10.10.0.0/16, App-B receives the request but has nowhere to send the answer, so the ping fails.

2. Why is the App-B SSH source the bastion's private IP? With ProxyJump, the bastion opens a new SSH connection to App-B, so App-B sees the bastion's private address (10.10.1.29 in my build) and not my laptop's public IP.

3. Why does Active peering not guarantee a successful ping? Active only means the link exists. The packet still needs routes in both subnets' route tables and a security group rule on the destination (and matching network ACL rules if ACLs are customised).

4. Can VPC B use VPC A's internet gateway over peering? No. Peering does not pass traffic on to a gateway in the other VPC, so VPC B stays without internet access. Peering is also one-to-one and not transitive: A-B plus A-C does not give B-C.

9. Troubleshooting checklist

Symptom	Check
Peering request stays Pending	Accept it from the accepter VPC's side
Ping times out	Routes on both sides, destination security group, network ACLs, and that the peering is Active
Ping works one way only	Missing return route in the other VPC's route table
Peering cannot be created	Overlapping CIDR ranges
SSH to the bastion fails	Your current public IP changed; update the /32 source in ewb-sg-bastion
SSH to a private host fails	Source in its security group must be the bastion's private IP; check key file permissions
Private host has no internet	Expected: there is no NAT or internet route for private subnets

10. Cost, quotas and limits

- Peering connections themselves have no hourly charge; data transferred across Availability Zones or Regions is billed.
- The main costs here are the three EC2 instances, their EBS volumes and the bastion's public IPv4 address, so I removed them as soon as testing was done.
- There is a default quota on active peering connections per VPC; this lab uses one.
- Pricing and quotas change, so check the current AWS pricing page before relying on these notes.

11. Choosing peering or an alternative

Option	Good for	Limitation
VPC peering	A small number of VPCs that need direct private connectivity	One-to-one, not transitive, no overlapping CIDRs
Transit Gateway	Many VPCs and on-premises networks through one hub	Hourly and per-GB cost, more setup
PrivateLink	Exposing one service privately to other VPCs	Service-level, not full network access

12. AWS CLI equivalent

I built this lab in the console. These commands show the same flow in the CLI; IDs are placeholders.

```
aws ec2 create-vpc --cidr-block 10.10.0.0/16
aws ec2 create-vpc --cidr-block 10.20.0.0/16
aws ec2 create-vpc-peering-connection --vpc-id <vpc-a> --peer-vpc-id <vpc-b>
aws ec2 accept-vpc-peering-connection --vpc-peering-connection-id <pcx>
aws ec2 create-route --route-table-id <rtb-a-private> \
    --destination-cidr-block 10.20.0.0/16 --vpc-peering-connection-id <pcx>
aws ec2 create-route --route-table-id <rtb-b-private> \
    --destination-cidr-block 10.10.0.0/16 --vpc-peering-connection-id <pcx>
```

13. Hardening and next steps

- Change the App-A ICMP rule from All ICMP to Echo request only.
- Use SSH ProxyJump with the key kept on my laptop, or Session Manager instead of a bastion.
- Restrict the bastion rule to a short-lived source IP and remove it when not in use.
- Turn on VPC Flow Logs to see accepted and rejected traffic.
- Define the whole lab in Terraform so it can be rebuilt and destroyed repeatably.

14. Cleanup plan

After testing I terminated ewb-bastion, ewb-app-a and ewb-app-b. I did not capture cleanup evidence. The remaining resources are removed in dependency order:

- 1. Terminate the three instances (done).
- 2. Delete the peering connection ewb-a-b-peer.
- 3. Delete the custom security groups.
- 4. Detach and delete the internet gateway.
- 5. Delete the subnets and custom route tables.
- 6. Delete both VPCs.
- 7. Delete the key pair.

15. What I learned

Peering alone does not give connectivity: the route tables on both sides and the destination's security group all have to allow the traffic. I also learned to keep the private key on my own laptop. When ProxyJump failed from Windows, I used a copy of the key on the bastion as a workaround, and next time I would fix the key file permissions and the SSH config instead.